

3.1- Client Server Basics

Task3- Web Communication in practice

Hypertext Transfer Protocol (Secure), abbreviated as HTTP(S), is a stateless client-server protocol used for the World Wide Web. This means that each request is processed independently, without the server retaining information about previous requests.

Although the protocol itself is stateless, modern websites and web applications implement mechanisms to introduce statefulness at the application level.

For example, when you log into a website using your credentials, the server creates a session identifier (often stored in a cookie or token) that is sent with each subsequent request.

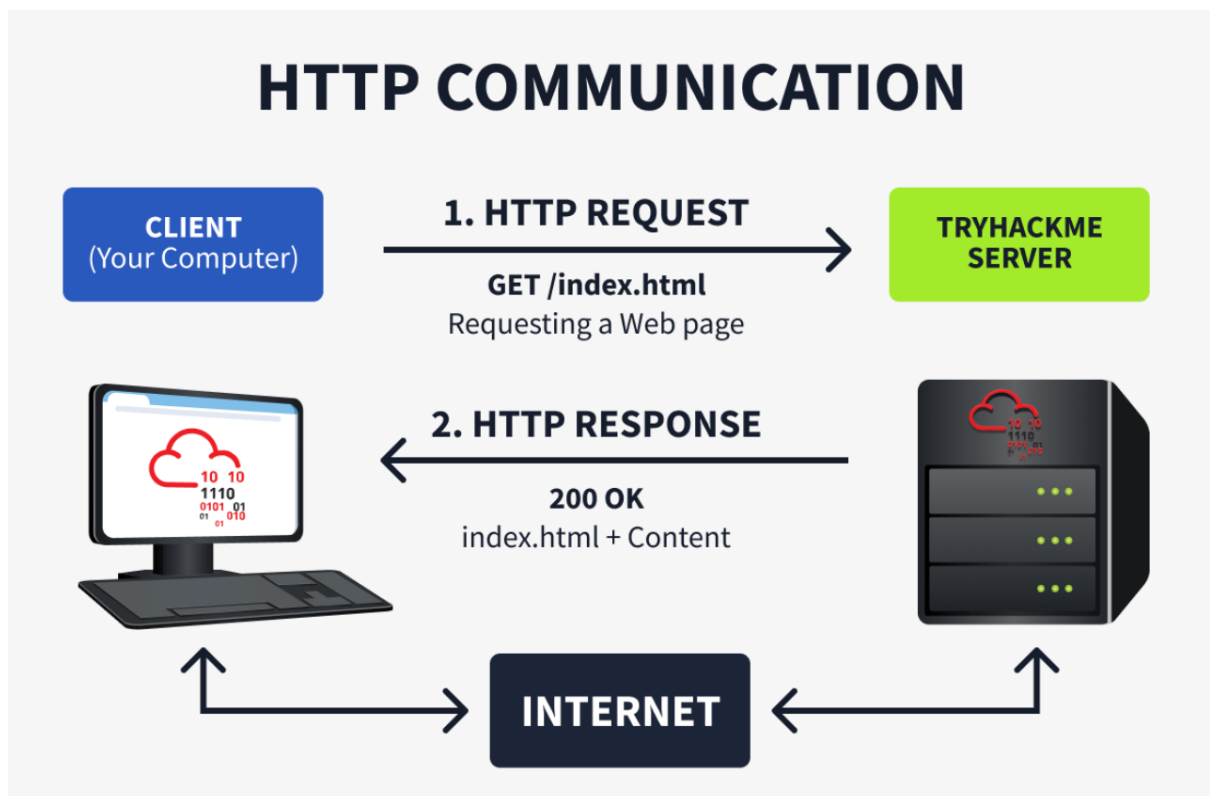
Without these mechanisms, you would need to authenticate again with every new request, because the server would have no memory of your previous login.

HTTP Commands

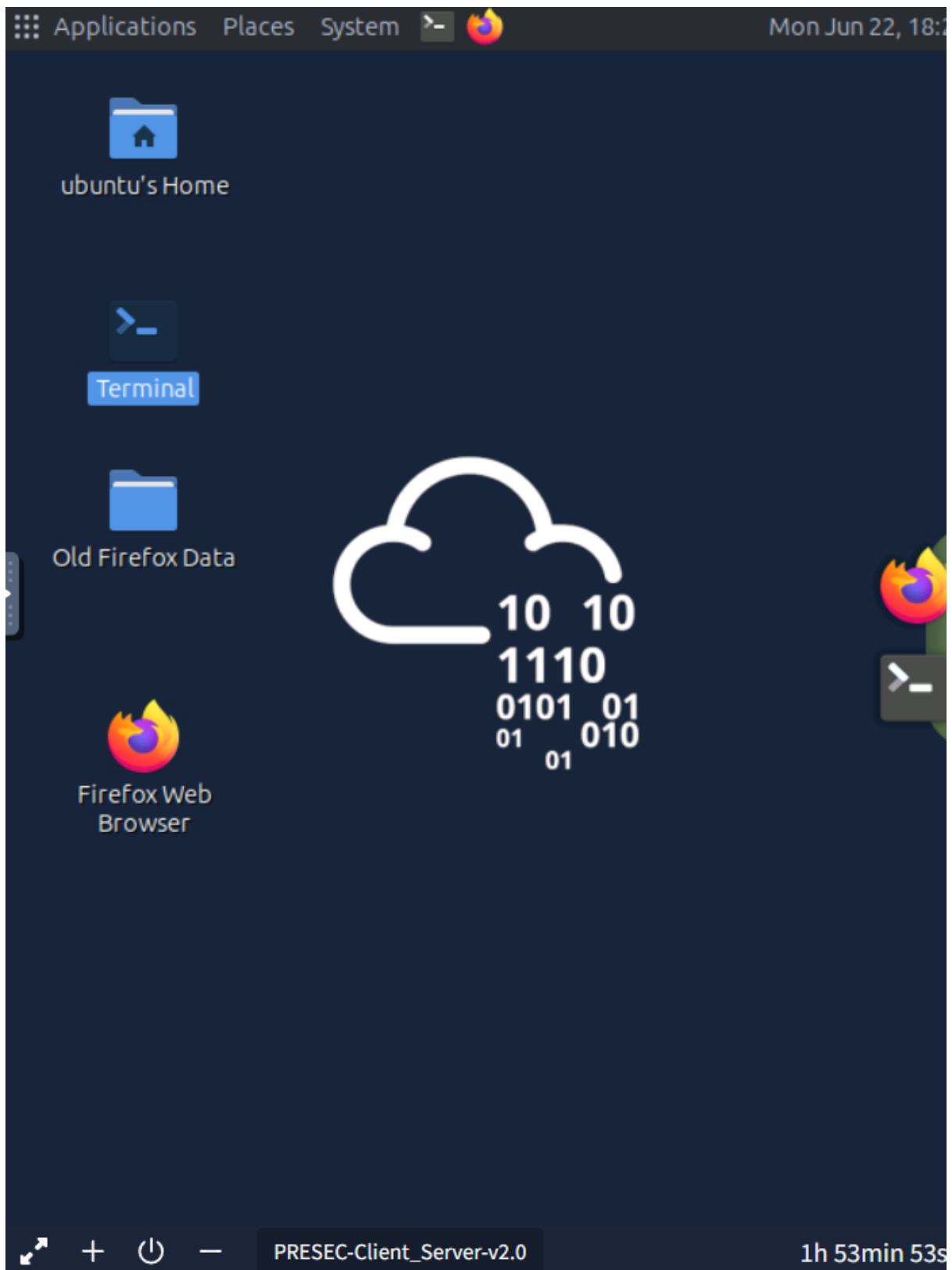
In the main specifications that define HTTP (also called Request for Comments, or RFC documents), there are 9 core commands. In HTTP lingo, we use the term method instead of command. Below you can see an overview of these methods:

- GET
 - POST
 - PUT
 - DELETE
 - PATCH
 - HEAD
 - OPTIONS
 - CONNECT
 - TRACE
- We will focus on discussing one of the two most common methods used: GET. We will approach this practically by examining the requests a browser sends when you navigate to a website.
 - Click on the "Start Lab Machine" button below. This will show a new window in split-screen where we can open a website and inspect a GET request. If the split-screen is not showing, click the blue "Start Split-screen" button at the top of the room.
 - GET

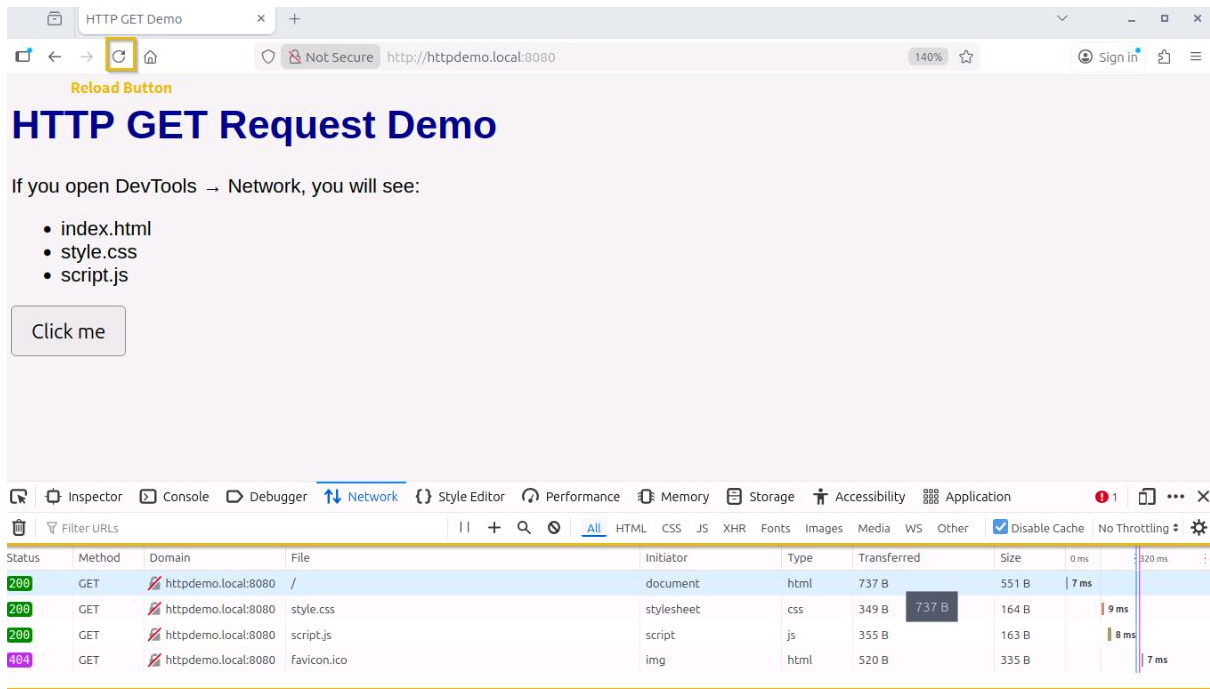
- The GET method is actually pretty straightforward. We can use this method to retrieve a resource from web server. For example, `GET https://tryhackme.com/index.php`. This request retrieves the TryHackMe website's homepage. You don't need to type in this request yourself. When you open a browser (this is the client) and type "https://tryhackme.com," the browser constructs the message behind the scenes using information you provide and other fields defined in the HTTP specifications. When the web server receives the request, it sends a response that includes a status code (Indicating the type of response) and the requested information. The image below shows the flow of this request.



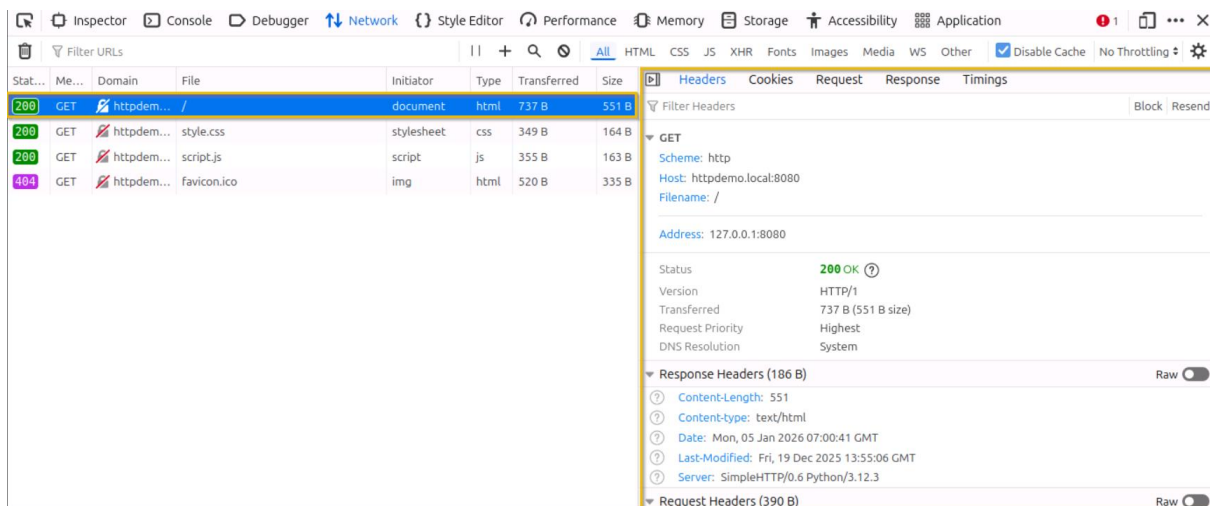
Let's have a look at an actual GET request and its response. Navigate to the Lab Machine that opened next to this screen, then click the Firefox icon on the Desktop. Once the browser is open, the web page `http://httpdemo.local:8080` should show. Proceed by pressing F12 or right-clicking in the browser window and selecting "Inspect". This will open the Firefox Developer Tools, which allows us to inspect, debug, and analyze web pages and traffic. Click on the "Network" tab as shown in the image below.



Open firefox web browser



Now reload the page by clicking the circular logo highlighted on the image above (next to where you type the URL). You should see multiple GET requests appearing in the Developer Tools window, under the Network tab. Click on the first entry as shown below to see more information.



In the right-hand panel, we can see more information about our GET request. We won't go into much detail, but let's have a look at some of these fields.

Scheme: Tells us which protocol was used: HTTP or HTTPS.

Host: Tells us the name of the host we request resources from.

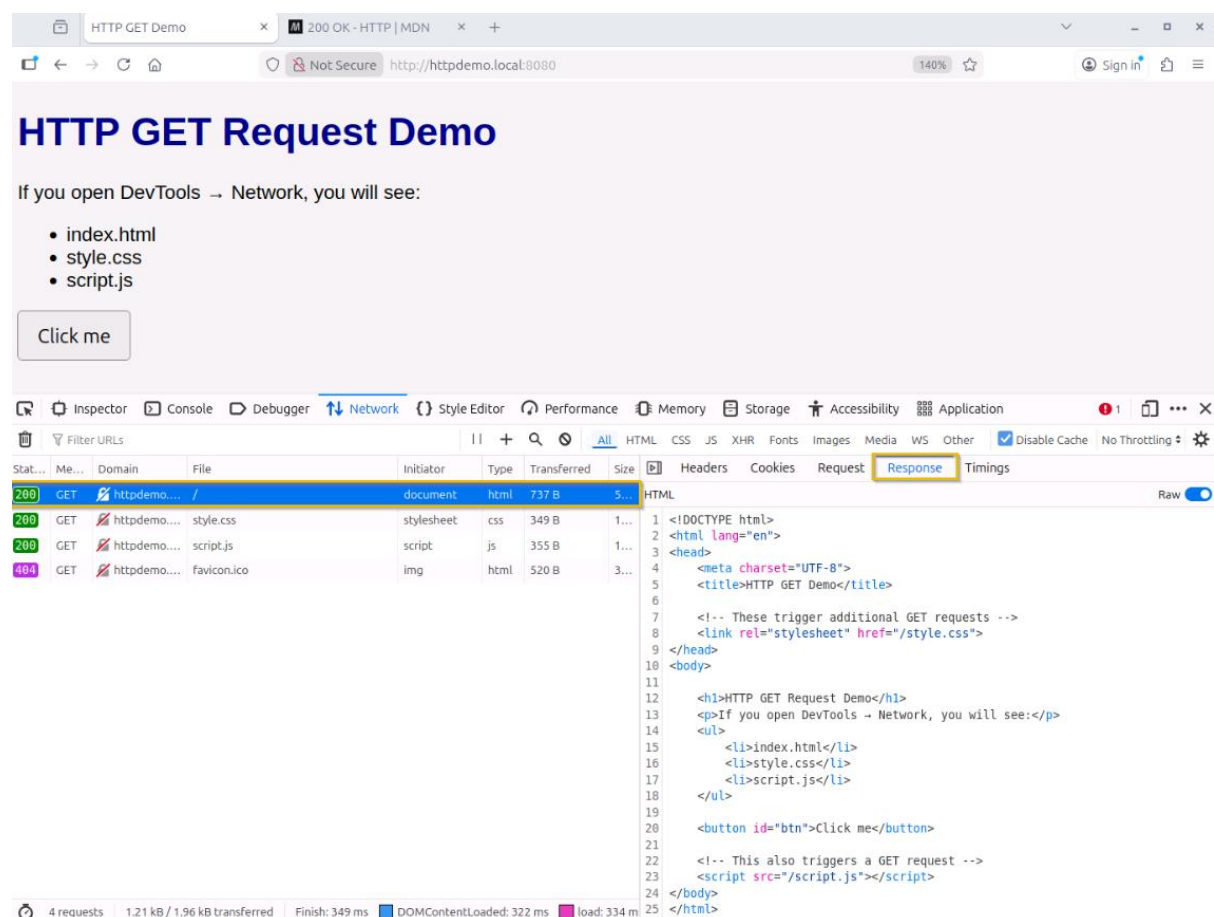
Filename: Indicates which file we requested from the host. In our request, this is "/", which actually translates to "index.html".

Address: Displays the IP address where the website is hosted. In our example, we are hosting the website on the same device. That's why the address 127.0.0.1 is shown.

Status: This field indicates whether the request was successful. In our example, we received a "200 OK" status, which means that the request was successful.

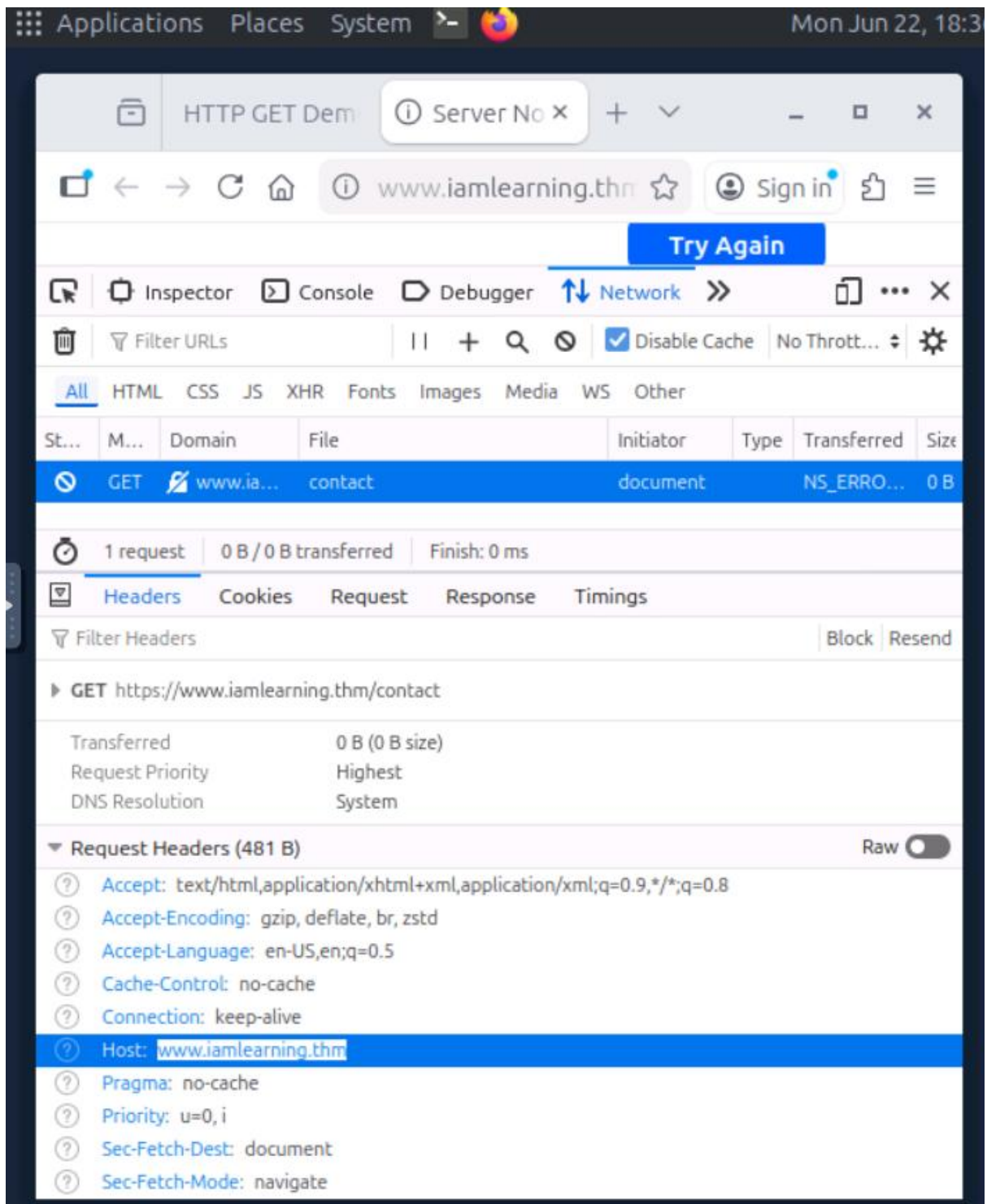
When a request is sent, we will get a response from the server. The response is divided into two parts: the response header and the response body. The response header contains metadata about the response, while the response body contains the requested content.

We can see this response body by clicking on the "Response" tab when displaying the details of a request. In our example, the response contains the index page of the requested website. The image below shows the content in HTML format.



Quiz:

What would be the host in the following URL? <https://www.iamlearning.thm/contact>



www.iamlearning.thm

2) What would be the scheme in the following URL? <https://www.iamlearning.thm/contact>

A) https